



Certified Kubernetes Administrator (CKA)

By Cloud Mentor Pro

Updated: 10/04/2026



Mock Exam - 3 (Wohoooo! Level Up!)

- *9 Questions*

Q1

1 2 3 4 5 6 7 8 9



Weight: 12

Create a new service account with the name `pvviewer`. Grant this Service account access to `list` all PersistentVolumes in the cluster by creating an appropriate cluster role called `pvviewer-role` and ClusterRoleBinding called `pvviewer-role-binding`.

Next, create a pod called `pvviewer` with the image: `redis` and serviceAccount: `pvviewer` in the default namespace.

- ServiceAccount: pvviewer
- ClusterRole: pvviewer-role
- ClusterRoleBinding: pvviewer-role-binding
- Pod: pvviewer
- Pod configured to use ServiceAccount pvviewer ?

Q1 - Answer

```
k create serviceaccount pvviewer
k api-resources -o wide
k create clusterrole pvviewer-role --resource=persistentvolumes --verb=list
k create clusterrolebinding pvviewer-role-binding --clusterrole=pvviewer-role
--serviceaccount=default:pvviewer

k run pvviewer --image=redis --dry-run=client -o yaml > 1_pod.yaml
vi 1_pod.yaml #Then "serviceName: pvviewer"
ka 1_pod.yaml

#Verify
k get pv --as=system:serviceaccount:default:pvviewer
```

Q2



1

2

3

4

5

6

7

8

9



Weight: 12

List the **InternalIP** of all nodes of the cluster. Save the result to a file **/root/CKA/node_ips**.

Answer should be in the format: **InternalIP of controlplane** <space> **InternalIP of node01** (in a single line)

● Task Completed

Q2 - Answer

```
k get node -o json #Xem format
```

```
k get node -o=jsonpath="{.items[*].status.addresses[?(@.type == 'InternalIP')].address}" >  
/root/CKA/node_ips
```

```
cat /root/CKA/node_ips #192.6.247.3 192.6.247.6
```

Q3

Task



1 2 3 4 5 6 7 8 9

Weight: 12

Create a pod called `multi-pod` with two containers.

Container 1, name: `alpha`, image: `nginx`

Container 2: name: `beta`, image: `busybox`, command: `sleep 4800`

Environment Variables:

container 1:

`name: alpha`

Container 2:

`name: beta`

- ☐ Pod Name: multi-pod
- ☐ Container 1: alpha
- ☐ Container 2: beta
- ☐ Container beta commands set correctly?
- ☐ Container 1 Environment Value Set
- ☐ Container 2 Environment Value Set

Q3 - Answer

```
k run multi-pod --image=busybox --command sleep 4800 --dry-run=client -o yaml > 3.yaml
```

```
vi 3.yaml #Edit yaml file
```

```
spec:
  containers:
  - command:
    - sleep
    - "4800"
    image: busybox
    name: beta
    env:
    - name: name
      value: beta
  - image: nginx
    name: alpha
    env:
    - name: name
      value: alpha
```

```
ka 3.yaml #pod/multi-pod created
```


Q4



1 2 3 4 5 6 7 8 9

Weight: 8

Create a Pod called `non-root-pod` , image: `redis:alpine`

`runAsUser: 1000`

`fsGroup: 2000`

- Pod non-root-pod fsGroup configured
- Pod non-root-pod runAsUser configured

Q4 - Answer

```
k run non-root-pod --image=redis:alpine --dry-run=client -o yaml > 4.yaml
```

```
vi 4.yaml #Edit yaml them "securityContext"
```

```
spec:
```

```
  securityContext:
```

```
    runAsUser: 1000
```

```
    fsGroup: 2000
```

```
  containers:
```

```
  - image: redis:alpine
```

```
    name: non-root-pod
```

Q5



1 2 3 4 5 6 7 8 9

Weight: 14

We have deployed a new pod called `np-test-1` and a service called `np-test-service`. Incoming connections to this service are not working. Troubleshoot and fix it. Create NetworkPolicy, by the name `ingress-to-nptest` that allows incoming connections to the service over port `80`.

Important: Don't delete any current objects deployed.

- Important: Don't Alter Existing Objects!
- NetworkPolicy: Applied to All sources (Incoming traffic from all pods)?
- NetWorkPolicy: Correct Port?
- NetWorkPolicy: Applied to correct Pod?

Q5 - Answer

```
controlplane ~ → k get netpol
NAME          POD-SELECTOR  AGE
default-deny  <none>       9m27s
```



```
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  status: {}
```

Default deny all ingress traffic

You can create a "default" ingress isolation policy for a namespace by creating a NetworkPolicy that selects all pods but does not allow any ingress traffic to those pods.

```
service/networking/network-policy-default-deny-ingress.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Thêm cái này sẽ thành allow-all-ingress

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-ingress
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
```



This ensures that even pods that aren't selected by any other NetworkPolicy will still be isolated for ingress. This policy does not affect isolation for egress from any pod.

Q5 - Answer

PROBLEM: Đang có 1 netpol ở default ns "default-deny-ingress"

SOLUTION: Tạo mới 1 netpol

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: ingress-to-nptest

namespace: default

spec:

podSelector:

matchLabels:

run: np-test-1

policyTypes:

 - Ingress

ingress:

 - **ports:**

 - **protocol:** TCP

port: 80

Q6



1 2 3 4 5 6 7 8 9



Weight: 12

Taint the worker node `node01` to be Unschedulable. Once done, create a pod called `dev-redis`, image `redis:alpine`, to ensure workloads are not scheduled to this worker node. Finally, create a new pod called `prod-redis` and image: `redis:alpine` with toleration to be scheduled on `node01`.

key: `env_type`, value: `production`, operator: `Equal` and effect: `NoSchedule`

- Key = `env_type`
- Value = `production`
- Effect = `NoSchedule`
- pod 'dev-redis' (no tolerations) is not scheduled on node01?
- Create a pod 'prod-redis' to run on node01

Q6 - Answer

6. Run the below command for solution:

▼ Details

```
kubectl taint node node01 env_type=production:NoSchedule
```



Deploy `dev-redis` pod and to ensure that workloads are not scheduled to this `node01` worker node.

```
kubectl run dev-redis --image=redis:alpine  
  
kubectl get pods -owide
```



Deploy new pod `prod-redis` with toleration to be scheduled on `node01` worker node.

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: prod-redis  
spec:  
  containers:  
    - name: prod-redis  
      image: redis:alpine  
  tolerations:  
    - effect: NoSchedule  
      key: env_type  
      operator: Equal  
      value: production
```



View the pods with short details:

```
kubectl get pods -owide | grep prod-redis
```



Q7



1 2 3 4 5 6 **7** 8 9

Weight: 8

Create a pod called `hr-pod` in `hr` namespace belonging to the `production` environment and `frontend` tier .
image: `redis:alpine`

Use appropriate labels and create all the required objects if it does not exist in the system already.

- hr-pod labeled with environment production?
- hr-pod labeled with tier frontend?

Q7 - Answer

```
kubectl create namespace hr
```

```
kubectl run hr-pod --image=redis:alpine --namespace=hr --labels=environment=production,tier=frontend
```

Q8



1

2

3

4

5

6

7

8

9

Weight: 8

A kubeconfig file called `super.kubeconfig` has been created under `/root/CKA`. There is something wrong with the configuration. Troubleshoot and fix it.

● Fix /root/CKA/super.kubeconfig

Q8 - Answer

```
vi /root/CKA/super.kubeconfig
```

→ Change the 2379 port to 6443 and run the below command to verify

```
kubectl cluster-info --kubeconfig=/root/CKA/super.kubeconfig
```

Q9 - Troubleshooting: kube-controller-manager.yaml



1 2 3 4 5 6 7 8 9

Weight: 14

We have created a new deployment called `nginx-deploy` . scale the deployment to 3 replicas. Has the replica's increased? Troubleshoot the issue and fix it.

● deployment has 3 replicas

Q9 - Answer

Lỗi ở "kube-controller-manager.yaml"

→ Change from "kube-**contro1ler**-manager" to "kube-**controller**-manager"